

Final Report

Fullstack Product Engineer Assessment

Candidate: KRISNANTO

Date: August 22, 2026

Repository: github.com/otnansirk/ai-interview-platform

1. Pull Request (GitHub)

I used the "Umbrella PR" strategy (Option B). I have one main branch called `feature/fitgap-monozukuri` that includes all changes from smaller sub-PR branches.

- **Umbrella PR:** <https://github.com/rakamindev/ai-interview-platform/pull/96>
- **Sub PRs:**
 - [#11 Feat/frontend ux polish](#)
 - [#10 feat: update LoginPage UI with new dashboard icon and enhanced branding](#)
 - [#9 Feat/frontend ux polish](#)
 - [#8 feat: enhance AssessorLayout with improved header and dropdown menu](#)
 - [#7 feat: enhance LoginPage UI with improved layout and error handling](#)
 - [#6 chore: add RefreshCw icon to PortfolioPage for enhanced UI options](#)
 - [#5 test: add specs for handling Gemini API timeout and log data leakage](#)
 - [#4 feat: frontend ux polish and responsive layout](#)
 - [#3 feat: frontend ux polish and fix data binding mismatch](#)
 - [#2 Chore/setup test harness](#)
 - [#1 chore: setup docker-compose arm64 and admin seeder](#)

2. Documentation

2.1 Product Understanding

I tested the app's workflow from beginning to end. This included logging in as an Assessor, creating assessments, inviting candidates, running AI interviews (WebSocket, Gemini), and viewing the final Portfolio and Fit/Gap Report.

The **Fit/Gap Report is the core product**. Recruiters read this report to decide if they should hire a candidate.

2.2 Industry Context (Indonesian Recruitment)

However, using AI for hiring is a big responsibility because it affects a candidate's future. We also must follow Indonesia's data privacy law (UU PDP). We must keep the candidate's private data (PII) safe and make sure it does not leak in the server logs.

2.3 Primary User & Empathy

2.3.1 Users: Assessors, Recruiters, and Hiring Managers

Hiring teams manage many candidates, sometimes dozens in one day. They use this product to get clear, objective data before deciding to interview someone or move them to the next stage.

2.3.2 Pain Points

When using the AI evaluation tool, recruiters need certainty. If the product shows unclear info, hides AI errors, or fails to load candidate data, it causes problems. It forces them to guess.

A wrong conclusion can unfairly affect a real candidate's future. Mistakes in the system make work harder for recruiters and can be life-changing for the people being evaluated.

2.4 Product Goal

To provide accurate, transparent, and trustworthy candidate assessment, while giving users confidence that the AI technology is working correctly.

3. Defining Problems & Gaps

I categorized my findings based on how serious they are:

P1 - API & Data Contract Errors (Broken Features)

No	Problem	Result / Impact	Status
1	ENV Ambiguity: APP_BASE_URL was used for both API and Invite Links. APP_BASE_URL was used for both Backend API and Frontend Invite Links.	Candidates received broken links (404).	Fixed / Resolved
2	Outdated Gemini Model: Deprecated models were configured. gemini-2.5-pro (deprecated) and gemini-3.5-flash-lite (not WebSocket compatible) were configured in application.yml	WebSocket connections were rejected and portfolio generation failed.	Fixed / Resolved

No	Problem	Result / Impact	Status
3	Schema Inconsistency: Frontend expected a string, but Backend sent an integer. Frontend used <code>required_level</code> (string), while Backend sent <code>expected_level</code> (integer).	The Fit/Gap table showed "undefined" values.	Fixed / Resolved
4	Type Mismatch: <code>ai_level</code> stored as integer in DB, but Frontend expected string.	Risk of fatal errors during future data manipulation.	Fixed / Resolved

P2 - UX & Quality Gaps (Features not yet made)

N	Problem	Result / Impact
1	Hardware Check Loop: No timeout on internet speed test.	UI freezes; candidates get stuck on the checking screen.
2	No Skeleton Loaders: Only simple spinners are used.	UX feels unprofessional while waiting for AI generation.
3	AI Text Overflow: Long AI text goes outside the cards.	Layout looks messy and is hard to read.
4	Mobile Layout: Action buttons stack incorrectly on small screens. Action buttons (PDF/JSON/Regenerate) stacked incorrectly on small screens.	Broken UI on mobile devices.
5	White Screen: No error handling if AI fails. Lack of error boundaries if Sidekiq or Gemini fails.	The app looks like it crashed without explanation.

P3 - Infrastructure Issues

No	Zeitwerk Autoloader Naming convention issues prevented booting.	Crash: CORS KeyError: Missing fallback for ENV variables.

No	Zeitwerk Autoloader Naming convention prevented booting.	Crash: issues	CORS KeyError: Missing fallback for ENV variables.

P0 - False Alarm (Not Broken)

The concern about crashes when candidates have no skills was already handled by the "not_assessed" mechanism. No fix needed.

4. Renovation Strategy & Decisions

4.1 Strategy: Single Source of Truth (Data Contract)

Evaluated Options:

Option	Approach	Pro	Con
A	Modify Backend to match Frontend format ("L3", required_level).	No changes required for Frontend.	Violates DB conventions; risky migration; creates new tech debt.
B	Update Frontend to follow Backend/DB standards.	Single source of truth (DB); no migration needed; consistent data contract. no migration needed; consistent data contract.	Frontend requires interface and component refactoring.
C	Create a transformation layer in the API serializer.	No changes required to DB or Frontend.	Increases complexity; leaky abstraction; harder to maintain.

Decision: Option B (Follow the Backend/DB standard). This is safer because the database is the single source of truth. Changing the Frontend is easier than risking a production database migration.

Acceptance Criteria:

- ai_level in TypeScript interface is type number, not string.
- ComparisonTable.tsx uses expected_level property, not required_level.

- No undefined values appear in the Fit/Gap table columns.

4.2 Strategy: Graceful Degradation (AI Resilience)

Option	Approach	Pro	Con
A	Allow Sidekiq to retry Gemini calls indefinitely.	Simple to implement.	Consumes RAM, stalls queues, poor UX (infinite loading).
B	Fallback Narrative + Error Retry UI	UX remains responsive; quantitative data is still served; users maintain control.	Qualitative narrative is not as high quality as Gemini's.
C	Strict timeout + immediate error display.	Fail fast approach.	Too aggressive; users lose all processed data.

Decision: Option B (Fallback Narrative + Error Retry). If Gemini times out, the system generates a basic report instead of crashing.

If Gemini times out, the system still generates a quantitative report (skill comparison table) with a generic fallback narrative. The UI displays Skeleton Loaders during generation and an Error Banner with a manual Retry button if it fails completely.

Acceptance Criteria:

- When Gemini times out, `FitGap::Engine` does NOT crash; it generates a fallback `overall_narrative`.
- UI displays table and paragraph Skeleton Loaders during generating status.
- UI displays Error Banner with a Retry button when status is failed.
- API Keys and Candidate PII (Personally Identifiable Information) NEVER appear in Rails logs.

4.3 Strategy: Dev vs. Production Separation

Option	Approach	Pro	Con
A	Directly modify Dockerfile (bundle config set frozen false).	Simple to implement.	Weakens production security; <code>Gemfile.lock</code> may change unintentionally.
B	Override <code>di</code> <code>docker-compose.yml</code>	Production Dockerfile remains frozen; local	Slightly more verbose in the compose file.

Option	Approach	Pro	Con
		development remains flexible.	
C	Create separate Dockerfiles for development and production.	Full isolation of environments.	Code duplication; increased maintenance burden.

Decision: Option B (Override in docker-compose.yml). This keeps production secure while allowing developers flexibility.

Default Dockerfile remains intact (Production Frozen mode). The command `bundle install` is injected specifically in the `docker-compose.yml` layer, providing flexibility for Developer Experience without compromising the production release architecture.

5. Implementation

5.1 Infrastructure, DevOps & Developer Experience

- Upgraded Docker Compose, injected Admin Seeder, fixed Zeitwerk and CORS issues, and set up a CI/CD pipeline.
- Local Development Environment (Developer Experience):
 - Provided a `docker-compose.yml` to simplify developer onboarding, allowing them to run the application locally without complex manual setup (Ruby/Postgres/Redis).
 - Optimized local Docker setup by using Alpine images for Postgres 16 and Redis 7 with ARM64 architecture support.
 - Added `shm_size: 256mb` to `docker-compose.yml` specifically for local development to prevent Out-of-Memory errors in the database when running test suites or large seed processes.

5.2 Frontend UX Polish

- Fixed data types, added skeleton loaders and error retry buttons, made the UI responsive for mobile, and fixed text overflow.

5.3 Testing & Verification

- I added Seeded Fault Tests to prove the tests work. I did this on a separate scratch branch named [seeded-fault-proof](#). I broke the logic on purpose (commit: [a65567b782173e0ae1339a7af5b73a06b4c53ef1](#)), confirmed the RSpec test failed, and then reverted it (commit: [92bce5b3c47a2916be2fd75471e137c098a19ec9](#)) to

show the visible history. I did not merge this branch to keep the main PR clean. I also filtered the server logs to hide candidate private data to follow UU PDP.

5.4 API Documentation & Quality Control

- I added Test-Driven Documentation (Rswag) to make the API documentation.

AI Verification Moment:

When I made the Swagger docs, my AI coding tool suggested using string for the ai_level data type. If I just followed the AI blindly, the data mismatch bug would still be there. I checked the real PostgreSQL database and saw the AI was wrong. I ignored the AI suggestion and typed integer instead. This makes the documentation correct and safe.

6. Screenshots

```
COOING-TEST/ai-interview-platform > docker compose exec api bundle exec rspec
seeded-fault-proof

Failures:

1) FitGap::EngineCall [SEEDED FAULT] when Gemini API times out does not leak candidate PII or API key in logs
Failure/Error: raise [:log_content2json, { include_question_candidate_name: session_candidate_name_present?
expected %i, [1820-00-23T00:43:37.551629 #331] 18000 -- : [N3] Narrative generation failed: Gemini API timeout...6-00-23T00:43:37.551629 #331] INFO -- : [N3] Fit/gap report generated: portfolio=187 vacancy=94% not to include "Larlene Willis"
# ./spec/services/fit_gap/engine_spec.rb:58:in `block (< levels) in ctup (required)'

2) FitGap::EngineCall [SEEDED FAULT] expected_level integrity produces correct expected_level values in skill comparisons
Failure/Error: expect(compet(expected_level)).to be_between(1, 5)
expected_level [Icomp] expected_level [I] for skill "Icomp[skill_label]": is out of valid range 1..5
expected_level 100 for skill "Ruby" is out of valid range 1..5
# ./spec/services/fit_gap/engine_spec.rb:88:in `block (< levels) in ctup (required)'
# ./spec/services/fit_gap/engine_spec.rb:79:in `each'
# ./spec/services/fit_gap/engine_spec.rb:79:in `block (< levels) in ctup (required)'

Top 3 slowest examples (0.475 seconds, 98.4% of total time):
  FitGap::EngineCall [SEEDED FAULT] when Gemini API times out does not leak candidate PII or API key in logs
    0.41849 seconds ./spec/services/fit_gap/engine_spec.rb:43
  FitGap::EngineCall [SEEDED FAULT] when Gemini API times out uses fallback narrative without crashing
    0.02917 seconds ./spec/services/fit_gap/engine_spec.rb:29
  FitGap::EngineCall [SEEDED FAULT] expected_level integrity produces correct expected_level values in skill comparisons
    0.02385 seconds ./spec/services/fit_gap/engine_spec.rb:54

Finished in 0.48115 seconds (files took 1.32 seconds to load)
1 examples, 2 failures

Failed examples:

rspec ./spec/services/fit_gap/engine_spec.rb:43 # FitGap::EngineCall [SEEDED FAULT] when Gemini API times out does not leak candidate PII or API key in logs
rspec ./spec/services/fit_gap/engine_spec.rb:54 # FitGap::EngineCall [SEEDED FAULT] expected_level integrity produces correct expected_level values in skill comparisons

Randomized with seed 29808

COOING-TEST/ai-interview-platform > |
```

Test Suite Results: Failed Tests on Branch :

<https://github.com/otnansirk/ai-interview-platform/tree/seeded-fault-proof>

```
COOING-TEST/ai-interview-platform > docker compose exec api bundle exec rspec
feature/fitgap-momozakuri

WARNING: Skipping key "SECRET_KEY_BASE", Already set in ENV.
WARNING: Skipping key "DB_HOST", Already set in ENV.
WARNING: Skipping key "DB_USERNAME", Already set in ENV.
WARNING: Skipping key "DB_PASSWORD", Already set in ENV.
WARNING: Skipping key "REDIS_URL", Already set in ENV.

Randomized with seed 18417

Top 2 slowest examples (0.40461 seconds, 98.3% of total time):
  FitGap::EngineCall [SEEDED FAULT] when Gemini API times out uses fallback narrative without crashing
    0.40460 seconds ./spec/services/fit_gap/engine_spec.rb:38
  FitGap::EngineCall [SEEDED FAULT] when Gemini API times out does not leak candidate PII or API key in logs
    0.02771 seconds ./spec/services/fit_gap/engine_spec.rb:43

Finished in 0.58793 seconds (files took 1.43 seconds to load)
2 examples, 0 failures

Randomized with seed 18417

COOING-TEST/ai-interview-platform > |
```

Test Suite Results: Passing Tests

AI Interview

Senior Fullstack Engineer

45 minutes

- This is a voice interview. Make sure you're in a quiet place.
- The AI will ask follow-up questions — there are no scripts.
- The session will last up to 45 minutes.
- Your mic will be active throughout. You can end anytime.

OS & browser	✓	Passed
Internet	✓	Passed
↓ 38.37 Mbps ↑ 88.5 Mbps 149 ms		
Microphone	✓	Passed
		40
Audio output	✓	Passed
Start Interview		

Interview Readiness Check: A comprehensive hardware and system diagnostic screen ensuring microphone, browser, and internet stability before the candidate begins the session.



Rakamin AI Interview

Enter your credentials to access the dashboard

Email

admin@rakamin.com

Password

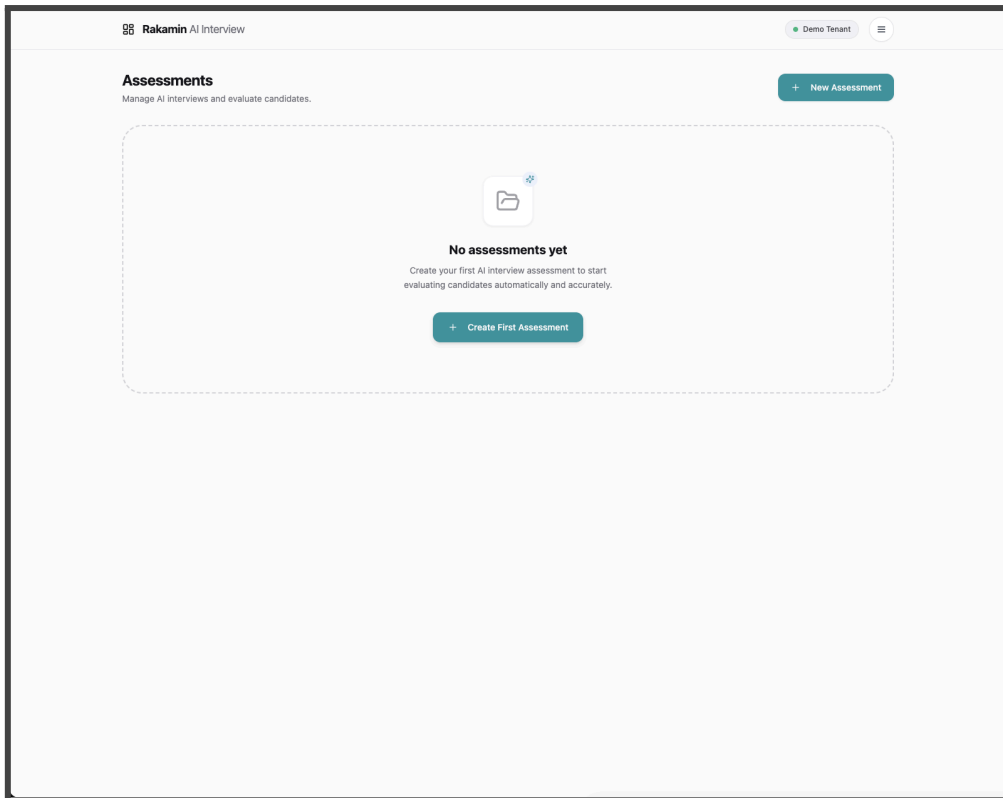
Forgot password?

••••••••

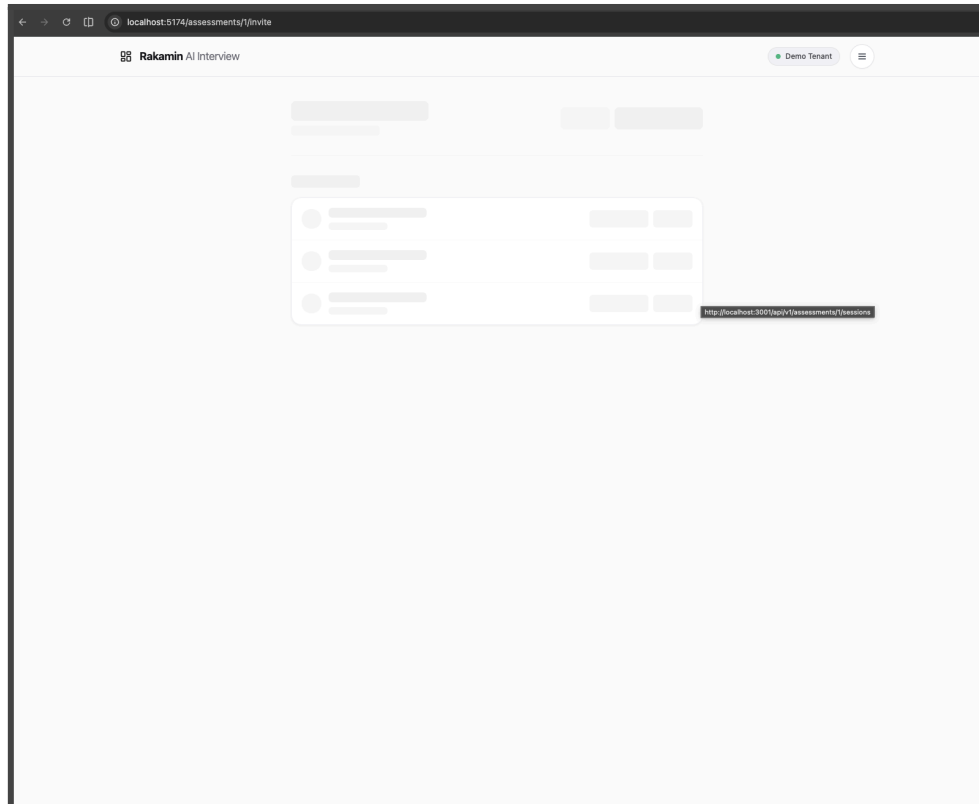
Sign in

By signing in, you agree to our [Terms of Service](#) and [Privacy Policy](#).

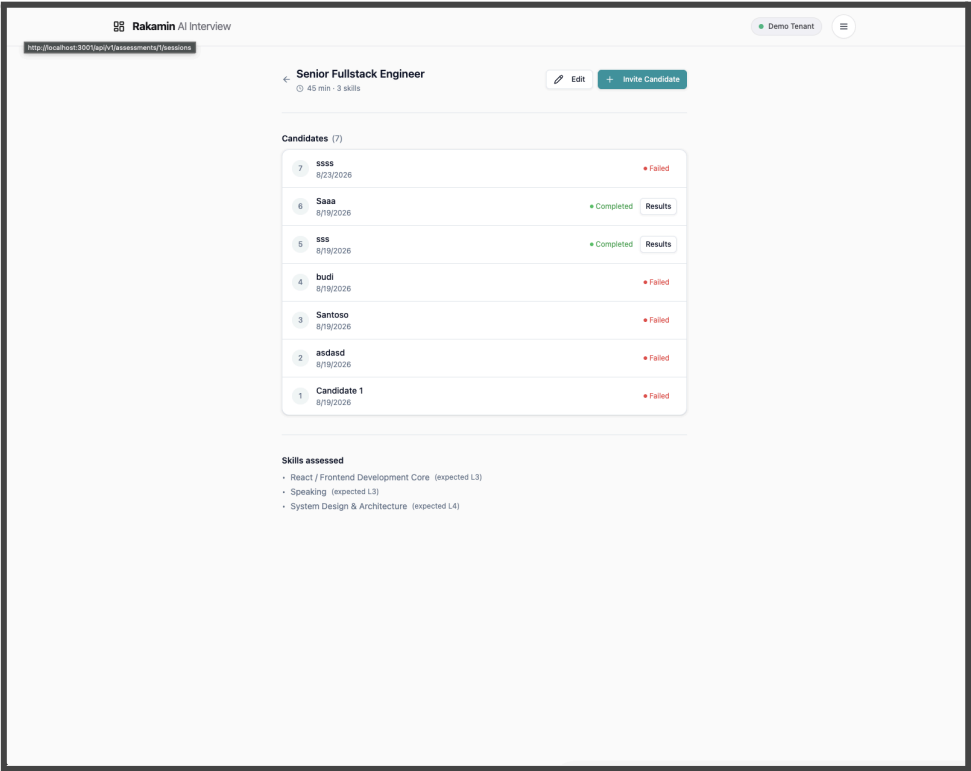
Assessor Login Interface: The secure authentication portal for administrators and assessors to access the platform's backend dashboard.



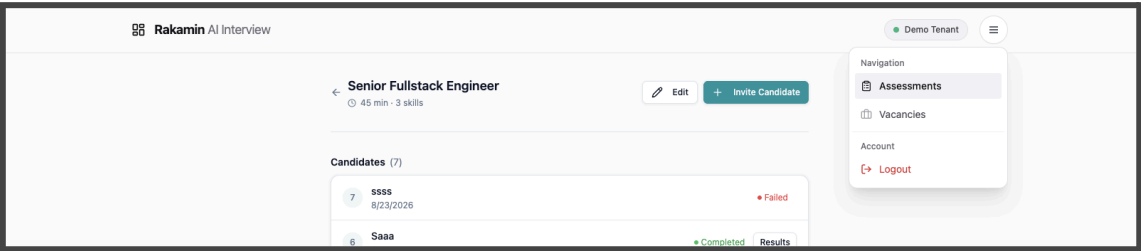
Empty Assessment State: *The primary dashboard view presented to assessors when no recruitment assessments have been configured yet.*



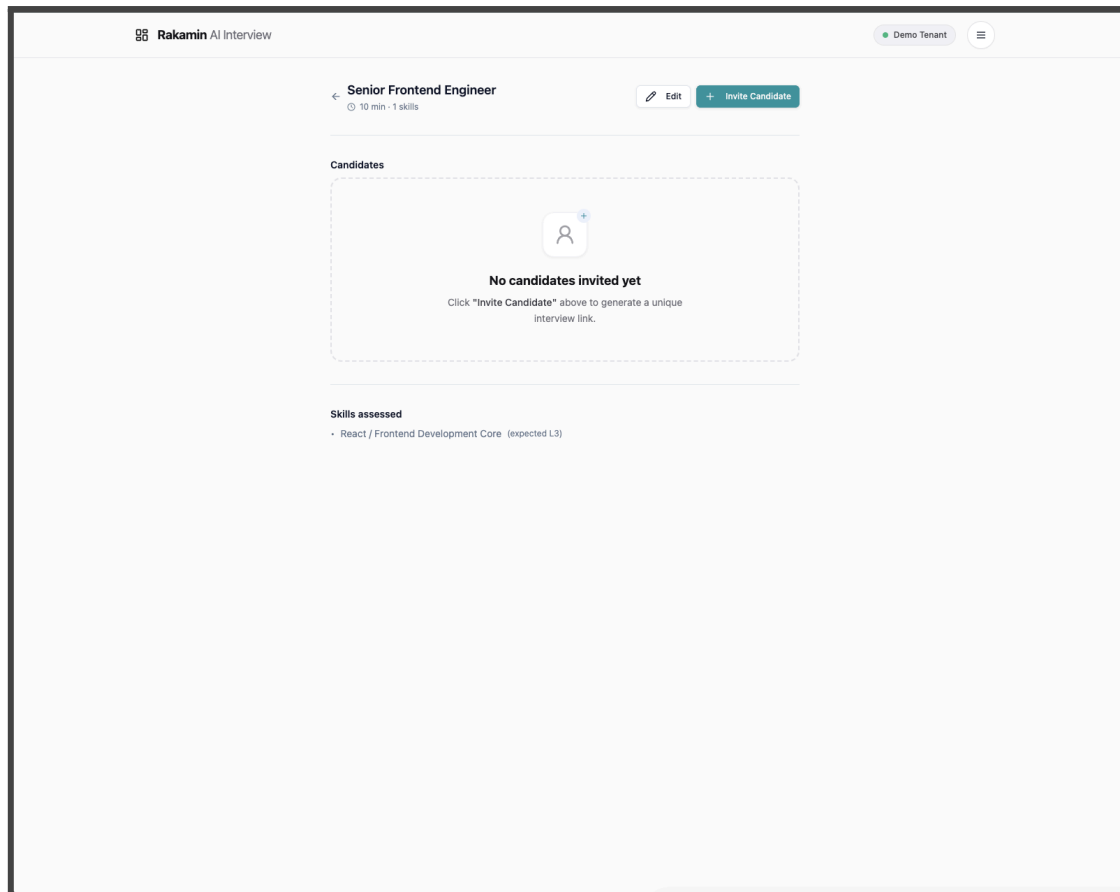
Data Loading State: *A skeleton loader implementation used to maintain visual consistency and professional UX during data fetching sequences.*



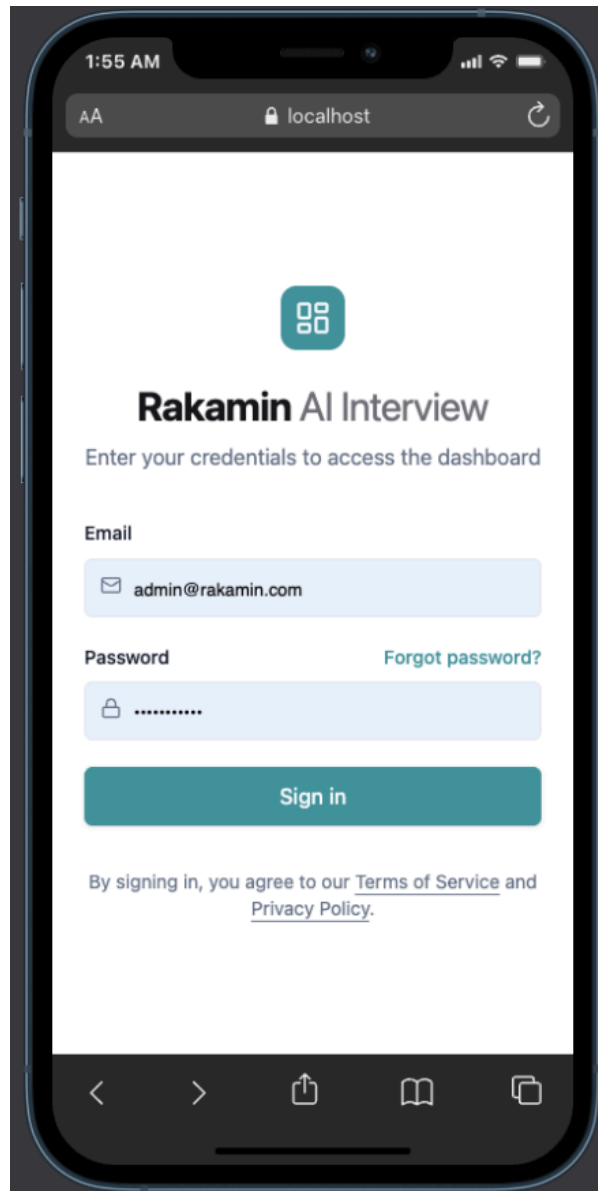
Assessment Overview Panel: Detailed view of a specific assessment showing the list of invited candidates and their respective evaluation statuses.



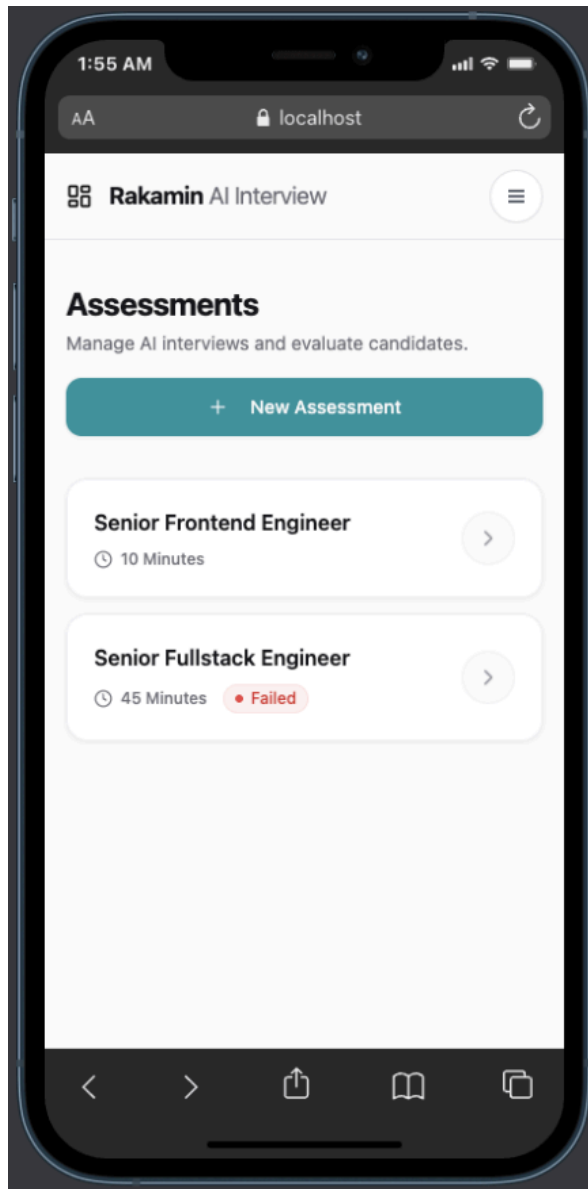
Global Navigation Drawer: The primary navigation menu providing quick access to assessments, vacancies, and account management settings.



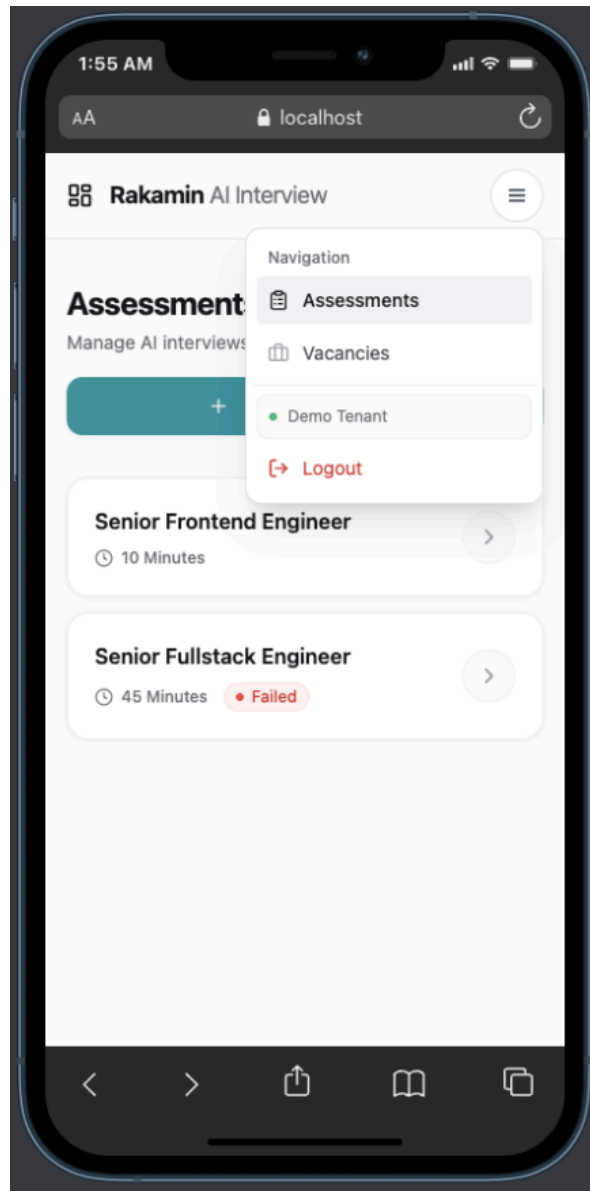
Candidate Invitation View: *The interface for generating unique interview links and managing candidate invitations for a new assessment.*



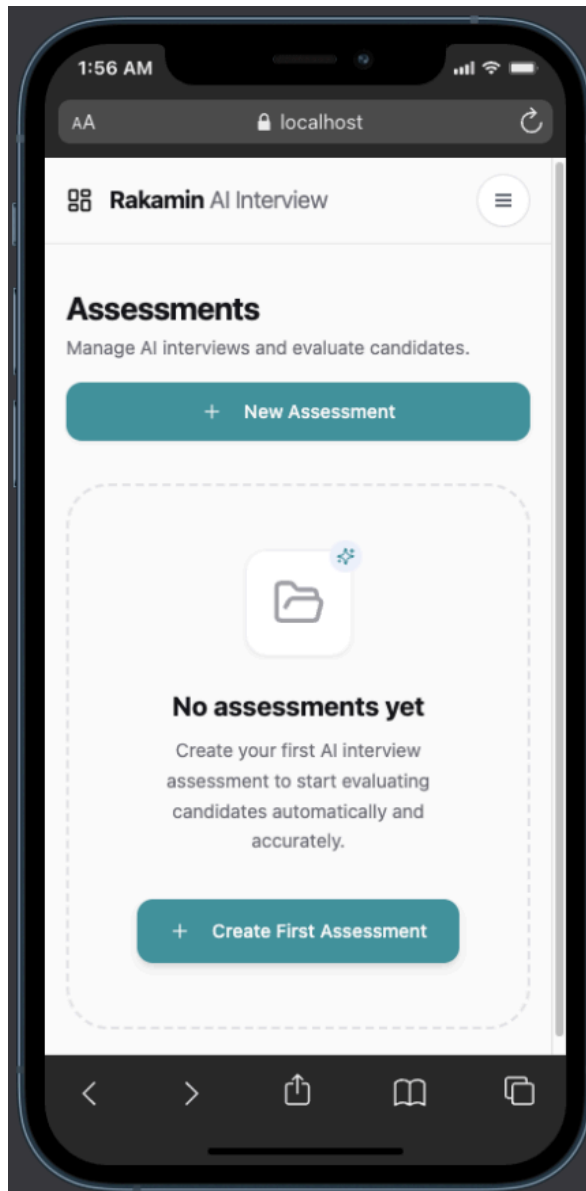
Mobile Login Optimization: *The responsive login interface specifically adapted for secure mobile access by assessors on the go.*



Mobile Dashboard View: *A streamlined version of the assessment dashboard optimized for vertical mobile screens.*



Mobile Navigation Menu: A touch-friendly navigation overlay ensuring full platform functionality on mobile devices.



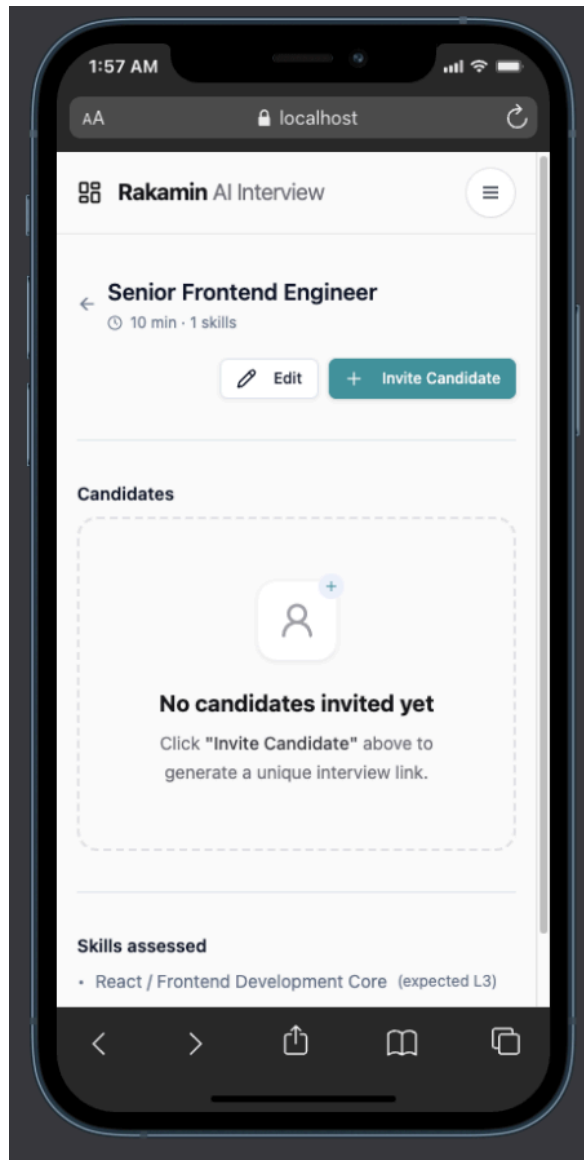
Mobile Empty State: *The intuitive "no data" placeholder adapted for the mobile application experience.*

The image shows a smartphone screen displaying a web application titled "Rakamin AI Interview". The status bar at the top shows the time as 1:56 AM, signal strength, Wi-Fi, and battery icons. The browser address bar shows "localhost". The form is designed for creating an assessment session and includes the following fields and controls:

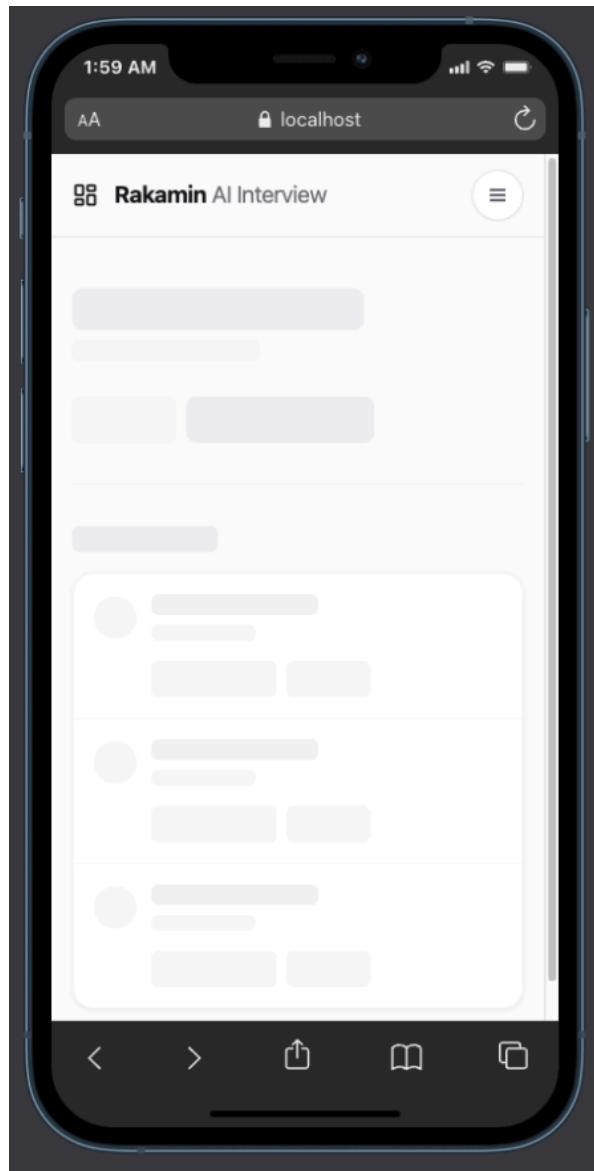
- Role title ***: A text input field containing "Senior Frontend Engineer".
- Session time limit ***: A dropdown menu currently set to "45 min".
- Interview language**: A dropdown menu currently set to "English".
- Skills to assess**: A section with a message "No skills added yet. Add at least one skill to continue." and two buttons: "+ Add from Skill Taxonomy" and "+ Add custom skill".
- Bottom actions**: Two buttons, "Cancel" and "Save & Create Session →", with the latter being highlighted in teal.

The bottom of the screen shows the standard iOS home indicator and a dock with navigation icons: back, forward, share, tabs, and a document icon.

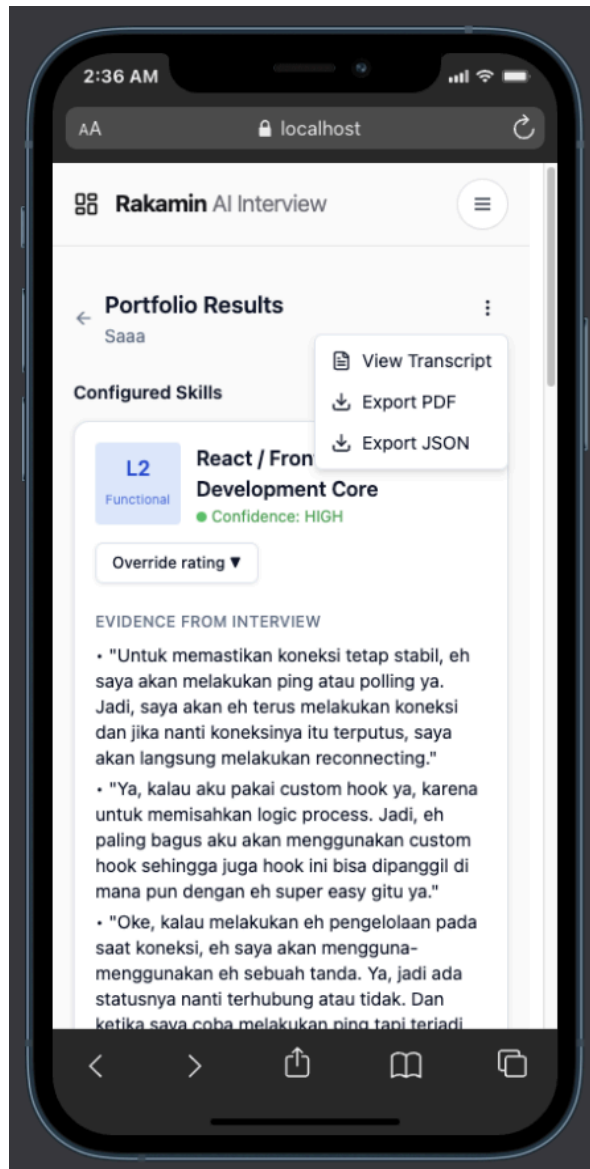
Mobile Assessment Setup: *The multi-step assessment creation form designed for ease of use on small touch screens.*



Mobile Candidate Management: *Mobile-optimized if no candidate list.*

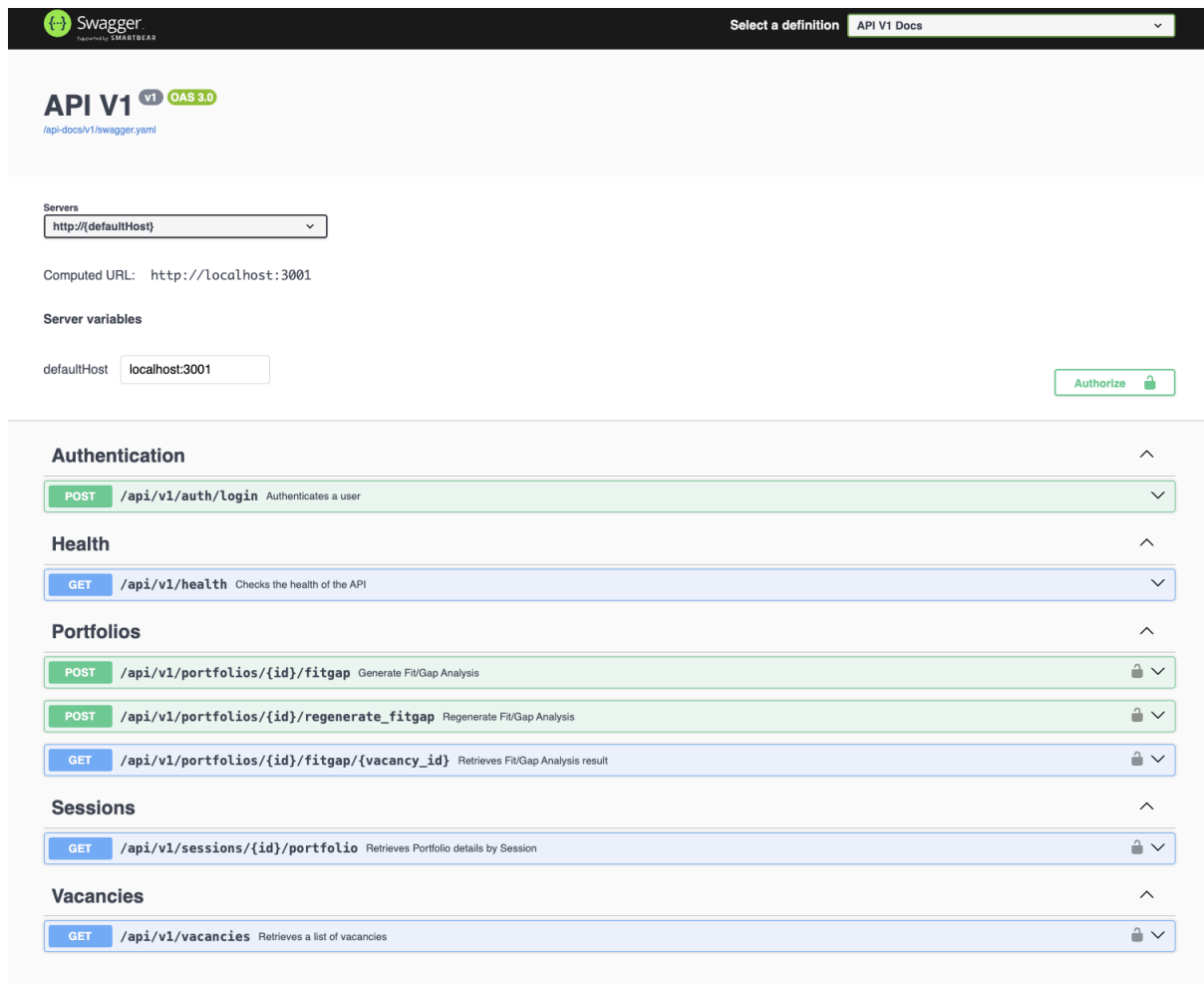


Mobile Loading Feedback: *Skeleton loading components tailored for mobile networks to improve perceived performance and interactivity.*



Portfolio Action Menu: A contextual menu allowing assessors to view transcripts or export detailed candidate reports in PDF and JSON formats.

7. Swagger UI — API Documentation (/api-docs):



Simplifies API integration for the frontend team by providing a single source of truth for data via Swagger.

8. Video Demo

(Upload your explanation video to Loom / YouTube Unlisted)

- **Video Link:** <https://youtu.be/QXMQ2EBmMlg>

9. Assumptions

Per the brief instructions ("If something is ambiguous, make reasonable assumptions"), here are the assumptions I noted:

1. **Admin Account:** Assumed admin@rakamin.com / password123 are standard development credentials.
2. **Swagger Scope:** Documented 7 core endpoints to prioritize the most important fixes.
3. **Gemini Model:** Assumed new models are compatible replacements for deprecated ones.

4. **Privacy:** Focused on preventing data leaks in logs, not complex encryption.